Assignment 04 Extra

Digital RTL Design

Sequential Circuit Design

Marwan Abdelfattah

Contents

1 Question 1 **2**
1.1 4-Bit Linear Feedback Shift Register (LFSR) . . . 2
1.1.1 Inputs . . . 2
1.1.2 Output . . . 2
1.1.3 Flip-Flop Specifications . . . 2
1.1.4 Testbench Requirements . . . 2
1.2 RTL Design . . . 3
1.3 Testbench . . . 4
1.4 Simulation Waveform . . . 4

2 Question 2 **5**
2.1 4-Bit Synchronous Counter with Asynchronous Active-Low Set . . . 5
2.1.1 Inputs . . . 5
2.1.2 Output . . . 5
2.2 RTL Design . . . 5
2.3 Testbench . . . 6
2.4 Simulation Waveform . . . 7

3 Question 3 **8**
3.1 Parameterized 8-Bit Shift Register . . . 8
3.1.1 Parameters . . . 8
3.1.2 Inputs . . . 8
3.1.3 Output . . . 8
3.1.4 Functional Requirements . . . 8
3.2 RTL Design . . . 9
3.3 Testbench 1: Right Shift by 2 Bits . . . 10
3.4 Simulation Waveforms . . . 11

1 Question 1

1.1 4-Bit Linear Feedback Shift Register (LFSR)

Implement a **4-bit Linear Feedback Shift Register (LFSR)** using Verilog HDL. The LFSR generates a pseudo-random sequence of numbers that repeats periodically, as illustrated in the provided circuit diagram.

1.1.1 Inputs

- `clk`: Clock input.
- `rst`: Active-high asynchronous reset.
- `set`: Active-high asynchronous set.

1.1.2 Output

- `out`: 4-bit LFSR output, where `out[3]` corresponds to Q4, `out[2]` corresponds to Q3, `out[1]` corresponds to Q2, and `out[0]` corresponds to Q1.

1.1.3 Flip-Flop Specifications

FF1:

- D input.
- Clock input.
- Active-high asynchronous set input.
- Output Q, which is set to 1 when set is asserted.

FF2, FF3, and FF4:

- D input.
- Clock input.
- Active-high asynchronous reset input.
- Output Q, which is reset to 0 when rst is asserted.

Note: The `rst` and `set` signals must be asserted simultaneously to guarantee correct initialization and operation of the LFSR.

1.1.4 Testbench Requirements

Create a testbench to verify the functionality of the LFSR.

- Verify the asynchronous reset functionality of FF2, FF3, and FF4.
- Verify the asynchronous set functionality of FF1.
- Use self-checking conditions to validate the control signals.

- Verify the generated pseudo-random sequence through QuestaSim simulation waveforms.
- Confirm that the generated sequence repeats according to the provided LFSR circuit.

1.2 RTL Design

```
module LFSR(
    input clk, rst, set,
    output [3:0] out
);
reg q1, q2, q3, q4;
always @(posedge clk or posedge rst) begin
    if(rst) begin
      q2 <= 0;
      q3 <= 0;
      q4 <= 0;
    end else begin
      q4 <= q3;
      q3 <= q2;
      q2 <= q1 ^ q4;
    end
end
always @(posedge clk or posedge set) begin
    if(set) begin
        q1 <= 1;
    end else begin
        q1 <= q4;
    end
end
assign out = {q4,q3,q2,q1};
endmodule
```

1.3 Testbench

```
module LFSR_tb();

reg clk, rst, set;
wire [3:0] out;

LFSR DUT (.*);
initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end
initial begin
    rst = 1;
    set = 1;
    @(negedge clk);
    if(out != 4'b0001) begin
      $display("Asynch Control Signals Error");
      $stop;
    end
    rst = 0;
    set = 0;
    repeat(1000) @(negedge clk);
    $stop;
end
endmodule
```

1.4 Simulation Waveform

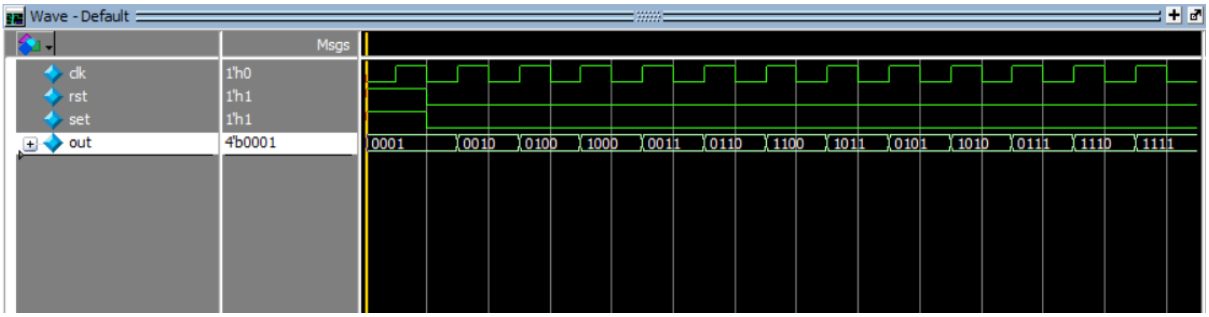


Figure 1: LFSR simulation waveform

2 Question 2

2.1 4-Bit Synchronous Counter with Asynchronous Active-Low Set

DImplement a **4-bit synchronous counter** using **behavioral modeling**. The counter should increment sequentially from 0 to 15.

The design should include an **asynchronous active-low set** input that sets all counter bits to `1`.

2.1.1 Inputs

- `clk`: Clock input.
- `set`: Asynchronous active-low set input that sets all counter bits to `1`.

2.1.2 Output

- `out`: 4-bit counter output.

The counter should operate synchronously with the clock and increment from:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \cdots \rightarrow 15$$

After reaching 15, the counter should wrap around to 0.

Use the **4-bit ripple counter implemented in Assignment 3 as a golden reference model**.

Create a **self-checking testbench** for the synchronous counter. The testbench should instantiate both the newly implemented synchronous counter and the ripple counter golden model and compare their outputs to verify that the two designs behave identically.

2.2 RTL Design

```
module counter (
    input clk, set,
    output reg [3:0] out
);
always @(posedge clk or negedge set) begin
    if(!set) begin
      out <= 4'b1111;
    end else begin
      out <= out + 1;
    end
end
endmodule
```

2.3 Testbench

```
module counter_tb ();
reg clk, rstn;
wire [3:0] out_struct, out_beh;

initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end

ripple_counter RIPPLE(clk, rstn, out_struct);
counter DUT (clk, rstn, out_beh);

integer i;
initial begin

rstn = 0;
@(negedge clk);
rstn = 1;

if(out_struct != out_beh) begin
  $display("Reset Function Error!");
  $stop;
end

for(i=0;i<100;i=i+1) begin
  @(negedge clk);
  if(out_struct != out_beh) begin
  $display("Error!!");
  $stop;
end
end
$stop;
end
initial begin
    $monitor("rstn=%b, clk=%b, out_struct=%b, out_beh=%b",
            rstn, clk, out_struct, out_beh);
end
endmodule
```

2.4 Simulation Waveform

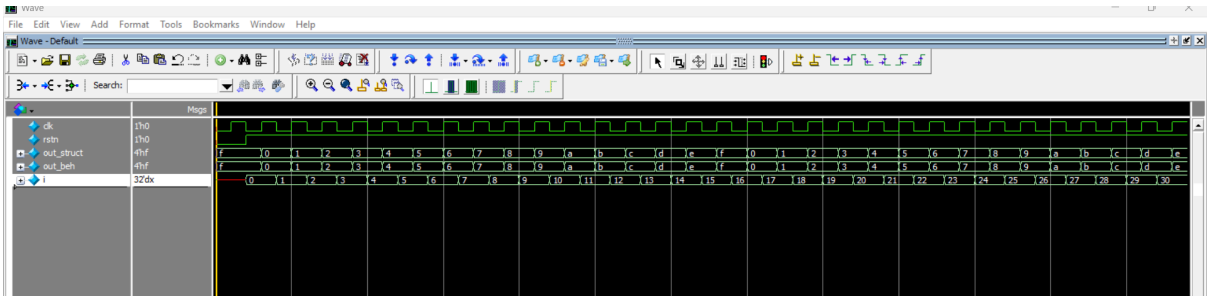


Figure 2: Simulation waveform

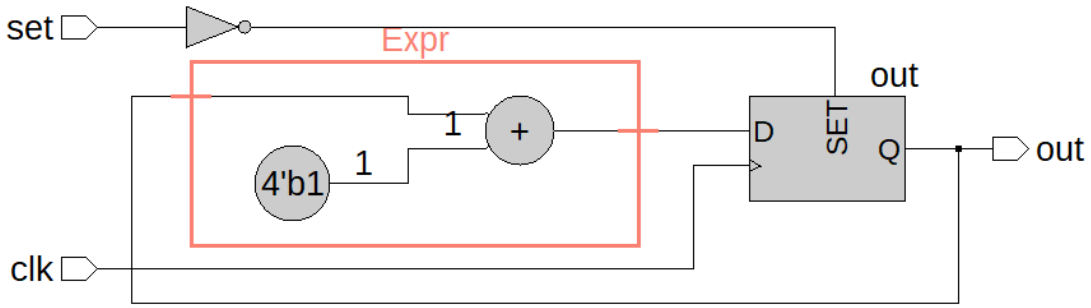


Figure 3: Schematic Diagram

3 Question 3

3.1 Parameterized 8-Bit Shift Register

Implement a **parameterized 8-bit shift register** using Verilog HDL. The design must support configurable shifting direction and shifting amount.

3.1.1 Parameters

- `SHIFT_DIRECTION`: Determines the direction of shifting. Valid values are `"LEFT"` and `"RIGHT"`. Default value: `"LEFT"`.
- `SHIFT_AMOUNT`: Determines the number of bits shifted per clock cycle. Valid values range from 1 to 7. Default value: `1`.

3.1.2 Inputs

- `clk`: Clock input.
- `rst`: Active-high asynchronous reset.
- `load`: Load control signal. When high, the register loads the input `load_value`.
- `load_value`: 8-bit parallel input value to be loaded into the register.

3.1.3 Output

- `PO`: 8-bit parallel output representing the current contents of the shift register.

3.1.4 Functional Requirements

- When `rst = 1`, the register must asynchronously reset to zero.
- When `load = 1`, the register must load the value of `load_value`.
- When `load = 0`, the register must shift according to the selected `SHIFT_DIRECTION` and `SHIFT_AMOUNT`.

3.2 RTL Design

```
module shift_reg #(
    parameter SHIFT_DIRECTION = "RIGHT",
    parameter SHIFT_AMOUNT = 2
) (
    input clk, rst, load,
    input [7:0] load_value,
    output reg [7:0] PO
);
always @(posedge clk or posedge rst) begin
    if (rst) begin
        PO <= 0;
    end else if (load) begin
        PO <= load_value;
    end else if (SHIFT_DIRECTION == "LEFT") begin
            PO <= {PO[7-SHIFT_AMOUNT:0],{SHIFT_AMOUNT{1'b0}}};

            //PO <= {{PO[7-SHIFT_AMOUNT:0]},{PO[7:8-SHIFT_AMOUNT]}}; //
    Circular Shift

    end else if (SHIFT_DIRECTION == "RIGHT") begin
            PO <= {{SHIFT_AMOUNT{1'b0}}, PO[7:SHIFT_AMOUNT]};

            //PO <= {{PO[SHIFT_AMOUNT-1:0]}, {PO[7:SHIFT_AMOUNT]}}; //Circular
     Shift
    end
end
endmodule
```

3.3 Testbench 1: Right Shift by 2 Bits

```
module shift_reg_tb ();

 reg clk, rst, load;
 reg [7:0] load_value;
 wire [7:0] PO;

 shift_reg #(.SHIFT_DIRECTION("RIGHT"), .SHIFT_AMOUNT(2)) DUT(.*);

 initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
 end


 initial begin
    load = 0;
    load_value = 0;
    rst = 1;
    @(negedge clk);
    rst = 0;

    if(PO != 0) begin
      $display("RST Error!!!");
      $stop;
    end

    load = 1;
    load_value = $random;
    @(negedge clk);
    load = 0;

    @(negedge clk);

    load = 1;
    load_value = $random;
    @(negedge clk);
    load = 0;
 end

initial begin
    $monitor("rst=%b, load=%b, load_value=%b, PO=%b",
                rst, load, load_value, PO);
end

endmodule
```

3.4 Simulation Waveforms

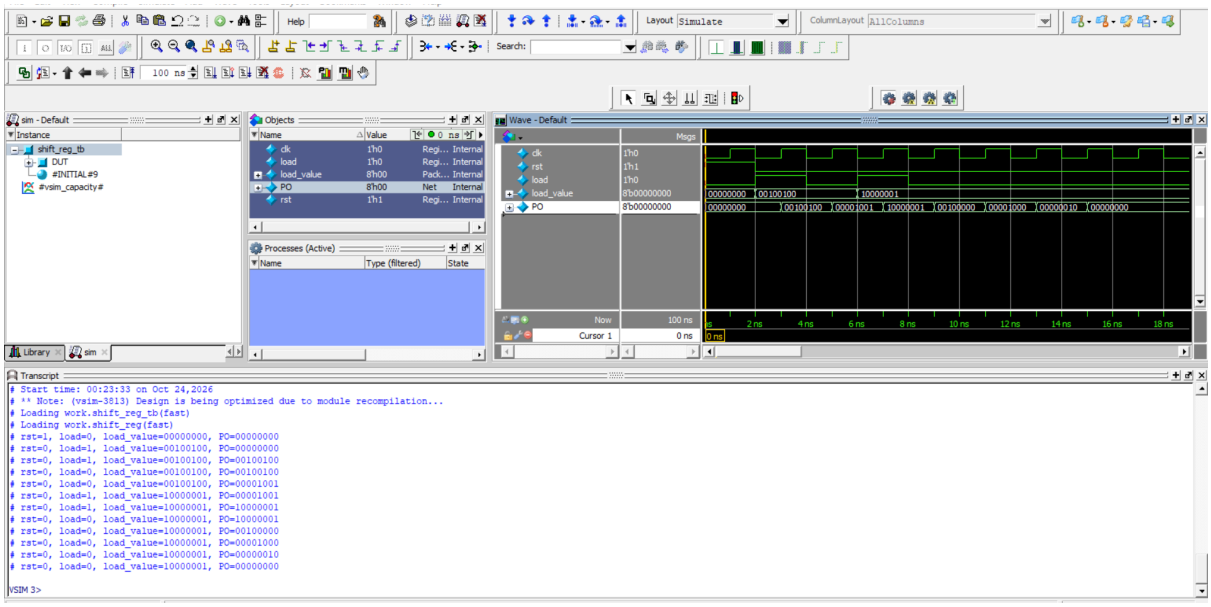


Figure 4: Shift register simulation waveform: Right shift by 2 bits